

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC

**ĐỀ TÀI: Hệ thống ưu tiên đèn giao thông cho xe khẩn cấp
sử dụng ESP32-CAM, YOLOv8 và YAMNet**

Môn học: NT532.Q21 - Công nghệ Internet of things hiện đại

Giảng viên hướng dẫn: Lê Minh Khánh Hội

Thực hiện bởi nhóm 08, bao gồm:

- | | | |
|-------------------------|----------|-------------|
| 1. Đỗ Gia Hân | 23520425 | Trưởng nhóm |
| 2. Bùi Nguyễn Tường Vân | 23521768 | Thành viên |
| 3. Trần Ngọc Thuý Vy | 23521837 | Thành viên |

Thời gian thực hiện: 04/2026 – 05/2026

MỤC LỤC

MỤC LỤC	2
TÓM TẮT.....	4
Chương I. TỔNG QUAN	5
1. Giới thiệu về đề tài	5
1.1. Mục tiêu đề tài	5
1.2. Phạm vi và giới hạn của đề tài.....	5
1.3. Bộ cục của đồ án.....	5
2. Cơ sở lý thuyết.....	6
2.1. Phân tích và lựa chọn linh kiện phần cứng.....	6
2.2. Lựa chọn nền tảng Cloud và Giao thức truyền thông	8
2.3. Các công cụ và thư viện phần mềm hỗ trợ	9
Chương II. THIẾT KẾ HỆ THỐNG	12
1. Kiến trúc hệ thống	12
1.1. Sơ đồ kiến trúc hệ thống.....	12
1.2. Luồng hoạt động hệ thống.....	13
2. Thiết kế tầng Thiết bị biên và Cảm biến	13
2.1. Cơ chế kích hoạt xác minh dữ liệu hình ảnh (Edge Triggering)	14
2.2. Quy trình xác minh âm thanh tại thiết bị biên	14
2.3. Cơ chế điều khiển đèn giao thông	15
3. Thiết kế tầng Xử lý Đám mây - Cloud Backend	15
3.1. Luồng xử lý hướng sự kiện của FastAPI Server	15
3.2. Thiết kế thuật toán phân tích AI kép tại FastAPI Server.....	16
3.3. Xây dựng logic xác thực đa yếu tố (Multi-modal Validation).....	16
4. Thiết kế giao diện web giám sát	17
Chương III. TRIỂN KHAI HỆ THỐNG	18
1. Lắp đặt phần cứng	18
2. Lập trình Firmware cho thiết bị biên.....	18
2.1. Mã nguồn ESP32-CAM xử lý Burst Capture và Upload dữ liệu.....	18
2.2. Mã nguồn ESP8266 điều khiển đèn giao thông và kết nối MQTT	22
3. Cấu hình và triển khai Cloud Backend.....	23

3.1. Cấu hình AWS EC2	23
3.2. Cấu hình Amazon S3	24
3.3. Cấu hình Amazon DynamoDB	25
4. Máy chủ AI (FastAPI AI Server).....	25
4.1. Huấn luyện mô hình AI nhận diện xe cứu thương	26
4.2. Xây dựng khối xử lý âm thanh còi ưu tiên	30
4.3. Lập trình logic xác thực đa yếu tố	31
5. Lập trình giao diện Web Dashboard	31
Chương IV. MÔ PHỎNG	32
1. Kịch bản 1: Không có xe cứu thương.....	32
2. Kịch bản 2: Phát hiện nghi ngờ xe cứu thương nhưng xác minh âm thanh không đạt	32
3. Kịch bản 3: Phát hiện và xác minh thành công xe cứu thương	32
Chương V. TỔNG KẾT	34
1. Kết quả đạt được.....	34
2. Hạn chế và khó khăn	35
3. Hướng cải tiến và phát triển	35
PHỤ LỤC	37
1. Bảng phân công công việc.....	37
2. Mã nguồn hệ thống	37
NGUỒN THAM KHẢO	38

TÓM TẮT

Trong quá trình đô thị hóa, mật độ phương tiện giao thông tại các giao lộ ngày càng gia tăng, kéo theo nguy cơ ùn tắc và làm giảm hiệu quả di chuyển của các phương tiện ưu tiên. Đối với xe cứu thương, mỗi phút chậm trễ trong quá trình di chuyển đều có thể ảnh hưởng trực tiếp đến việc cấp cứu và an toàn của người bệnh. Vì vậy, việc nghiên cứu và xây dựng một hệ thống ưu tiên đèn giao thông cho xe khẩn cấp là hết sức cần thiết, giúp hỗ trợ công tác cứu hộ, cứu nạn.

Xuất phát từ nhu cầu đó, đề tài xây dựng một hệ thống ưu tiên đèn giao thông thông minh cho xe cứu thương sử dụng kiến trúc Edge-Cloud kết hợp xử lý đa phương thức (Multimodal AI). Hệ thống sử dụng ESP32-CAM để thu nhận hình ảnh giao thông theo thời gian thực, kết hợp microphone INMP441 để thu âm thanh môi trường. Dữ liệu được gửi lên máy chủ AI triển khai trên AWS EC2 để thực hiện nhận diện xe cứu thương bằng mô hình YOLOv8 và xác minh còi ưu tiên bằng mô hình YAMNet.

Chương I. TỔNG QUAN

1. Giới thiệu về đề tài

1.1. Mục tiêu đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống ưu tiên đèn giao thông thông minh có khả năng phát hiện, xác minh và xử lý xe cứu thương trong thời gian thực, từ đó hỗ trợ điều khiển tín hiệu giao thông một cách tự động và hợp lý.

Đề tài tập trung vào các nội dung cụ thể sau: Thu thập hình ảnh giao thông bằng ESP32-CAM theo chu kỳ định kỳ, sử dụng mô hình YOLOv8 để nhận diện xe cứu thương từ ảnh chụp; thu nhận âm thanh môi trường bằng microphone INMP441, áp dụng mô hình YAMNet để phân tích và xác minh âm thanh còi ưu tiên, xây dựng cơ chế kết hợp kết quả từ hình ảnh và âm thanh để đưa ra quyết định cuối cùng, đồng thời triển khai cơ chế truyền thông thời gian thực giữa các thành phần thông qua MQTT.

1.2. Phạm vi và giới hạn của đề tài

Đề tài tập trung nghiên cứu và triển khai một mô hình thử nghiệm cho hệ thống ưu tiên đèn giao thông tại một giao lộ mô phỏng. Trong phạm vi thực hiện, hệ thống chỉ xét đến một loại phương tiện ưu tiên là xe cứu thương và thực hiện điều khiển đèn giao thông theo một kịch bản ưu tiên tuyến tính giữa hai làn xe.

Hệ thống được xây dựng theo kiến trúc Edge-Cloud, trong đó ESP32-CAM đóng vai trò thiết bị thu thập dữ liệu và ESP8266 đảm nhiệm chức năng điều khiển đèn giao thông. Máy chủ AI triển khai trên AWS EC2 thực hiện toàn bộ quá trình nhận diện, suy luận và ra quyết định. Hệ thống có thêm dashboard giám sát để theo dõi trạng thái hoạt động và lịch sử nhận diện.

1.3. Bố cục của đồ án

Đồ án được trình bày theo các chương với nội dung được sắp xếp theo trình tự từ tổng quan đến triển khai và đánh giá kết quả.

Chương 1: Tổng quan trình bày lý do chọn đề tài, mục tiêu nghiên cứu, phạm vi và giới hạn của hệ thống, đồng thời giới thiệu các cơ sở lý thuyết và công nghệ nền tảng được sử dụng trong quá trình xây dựng đề tài.

Chương 2: Thiết kế hệ thống trình bày kiến trúc tổng thể của hệ thống, mô hình hoạt động, luồng xử lý dữ liệu giữa các thành phần, cũng như thiết kế phần cứng và phần mềm cho từng khối chức năng.

Chương 3: Triển khai hệ thống mô tả quá trình hiện thực hóa đề tài, bao gồm triển khai phần cứng, xây dựng chương trình cho ESP32-CAM và ESP8266, phát triển máy chủ AI trên EC2, tích hợp mô hình nhận diện và xây dựng cơ chế truyền thông giữa các thành phần.

Chương 4: Mô phỏng và đánh giá trình bày kết quả kiểm thử hệ thống trong các tình huống khác nhau, đánh giá độ chính xác nhận diện, khả năng phản hồi thời gian thực, mức độ ổn định và hiệu quả vận hành của toàn hệ thống.

Chương 5: Tổng kết tổng kết những kết quả đã đạt được, chỉ ra các ưu điểm và hạn chế còn tồn tại, đồng thời đề xuất các hướng phát triển trong tương lai nhằm nâng cao chất lượng và khả năng ứng dụng thực tế của hệ thống.

2. Cơ sở lý thuyết

2.1. Phân tích và lựa chọn linh kiện phần cứng

2.1.1. Vi điều khiển ESP32 và mô-đun ESP32-CAM

ESP32 là dòng vi điều khiển do Espressif phát triển, được tích hợp WiFi và Bluetooth, có khả năng xử lý tốt trong các ứng dụng IoT. Nhờ hỗ trợ kết nối mạng không dây, tốc độ xử lý tương đối cao và khả năng tích hợp nhiều giao thức giao tiếp ngoại vi, ESP32 thường được sử dụng trong các hệ thống giám sát, điều khiển và thu thập dữ liệu thời gian thực.

Trong đồ án này, mô-đun ESP32-CAM được lựa chọn làm thiết bị biên chính. ESP32-CAM là phiên bản mở rộng của ESP32, được tích hợp camera OV2640, cho phép chụp ảnh và truyền dữ liệu hình ảnh qua WiFi.

Trong hệ thống, ESP32-CAM đảm nhận các nhiệm vụ chính sau:

- Chụp ảnh giao thông theo chu kỳ định kỳ.
- Gửi ảnh lên máy chủ AI để nhận diện xe cứu thương.
- Nhận lệnh xác minh từ Cloud thông qua MQTT.
- Kích hoạt chế độ chụp nhanh nhiều ảnh liên tiếp khi cần xác minh.
- Thu âm thanh môi trường từ microphone INMP441 trong giai đoạn xác minh.
- Gửi dữ liệu hình ảnh và âm thanh lên Cloud để xử lý.



Hình 1. Module ESP32-CAM

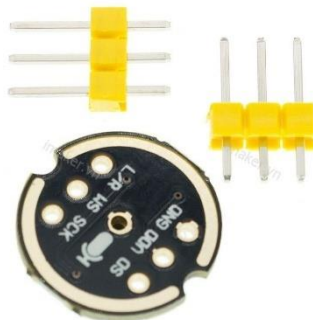
2.1.2. Cảm biến âm thanh INMP441 và cơ chế thu âm

INMP441 là microphone MEMS kỹ thuật số, hoạt động theo giao tiếp I2S. So với các loại microphone analog, INMP441 có ưu điểm là khả năng chống nhiễu tốt hơn, chất lượng tín hiệu ổn định hơn và dễ tích hợp với vi điều khiển nhúng có hỗ trợ I2S như ESP32.

Trong đề tài, microphone INMP441 được sử dụng để thu nhận âm thanh môi trường xung quanh giao lộ. Dữ liệu âm thanh này chỉ được ghi và gửi lên máy chủ khi hệ thống phát hiện nghi ngờ có xe cứu thương, nhằm phục vụ bước xác minh thứ hai. Điều này giúp giảm tải cho hệ thống và tránh việc ghi âm liên tục khi không cần thiết.

Cơ chế hoạt động của microphone trong hệ thống gồm các bước:

- ESP32-CAM nhận lệnh xác minh từ Cloud.
- Thiết bị kích hoạt microphone INMP441.
- Âm thanh môi trường được ghi trong khoảng thời gian ngắn.
- Dữ liệu âm thanh được chuyển thành file WAV để gửi lên máy chủ AI.
- Máy chủ sử dụng mô hình nhận diện âm thanh để kiểm tra sự xuất hiện của còi xe cứu thương.



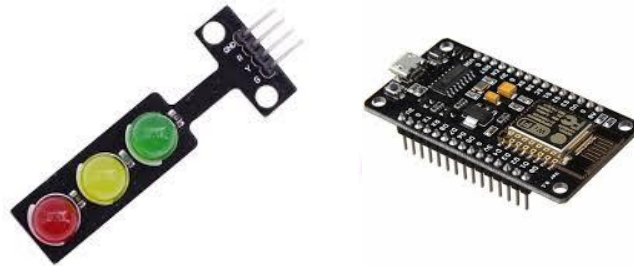
Hình 2. Cảm biến INMP441

2.1.3. Linh kiện phụ trợ: đèn LED, mạch nguồn và ESP8266

Để mô phỏng, hệ thống sử dụng các đèn LED đại diện cho tín hiệu đèn giao thông hai làn đường. Cụm đèn này gồm ba trạng thái cơ bản: đỏ, vàng và xanh. Các đèn được điều khiển bởi vi điều khiển ESP8266 thông qua các chân xuất tín hiệu số.

ESP8266 là vi điều khiển có tích hợp WiFi, thường được dùng trong các ứng dụng IoT đơn giản nhờ chi phí thấp và khả năng kết nối mạng ổn định. Trong đề tài, ESP8266 đóng vai trò bộ chấp hành, nhận lệnh điều khiển từ server và thực hiện thay đổi trạng thái đèn giao thông.

Bên cạnh đó, hệ thống còn cần nguồn cấp ổn định cho ESP32-CAM, ESP8266 và các đèn LED. Việc cấp nguồn phù hợp là yếu tố quan trọng để đảm bảo thiết bị hoạt động liên tục, tránh sụt áp hoặc reset trong quá trình truyền dữ liệu và xử lý tín hiệu.



Hình 3. ESP8266 và đèn LED

2.2. Lựa chọn nền tảng Cloud và Giao thức truyền thông

2.2.1. Nền tảng AWS EC2, S3 và DynamoDB

AWS EC2 là dịch vụ máy chủ ảo của Amazon Web Services, cho phép triển khai các ứng dụng web, API và mô hình AI trên môi trường điện toán đám mây với khả năng tùy biến cao. Trong đề tài, EC2 được sử dụng để triển khai máy chủ FastAPI phục vụ tiếp nhận dữ liệu từ ESP32-CAM, chạy mô hình nhận diện và điều phối quá trình xác minh phương tiện khẩn cấp.

Bên cạnh EC2, hệ thống còn sử dụng AWS S3 và DynamoDB. AWS S3 được dùng để lưu trữ ảnh và file âm thanh được gửi lên từ ESP32-CAM, phục vụ việc xem lại dữ liệu và minh chứng kết quả nhận diện. DynamoDB được dùng để lưu lại lịch sử các sự kiện nhận diện, bao gồm thời gian, trạng thái, kết quả phát hiện, mức tin cậy và đường dẫn dữ liệu.



Hình 4. Các dịch vụ AWS

2.2.2. Giao thức MQTT

MQTT (Message Queuing Telemetry Transport) là giao thức truyền thông nhẹ được thiết kế cho các hệ thống IoT có băng thông hạn chế hoặc yêu cầu độ trễ thấp. Giao thức này hoạt động theo mô hình publish-subscribe, trong đó thiết bị gửi bản tin lên một chủ đề cụ thể và các thiết bị đăng ký chủ đề đó sẽ nhận được bản tin tương ứng.

Trong hệ thống của đề tài, MQTT được sử dụng để truyền lệnh điều khiển từ Cloud đến ESP32-CAM và ESP8266. Cụ thể, khi Cloud phát hiện khả năng xuất hiện xe cứu thương, máy chủ sẽ gửi lệnh `prepare_verify` để yêu cầu thiết bị chuyển sang chế độ xác minh. Sau khi kết quả xác minh cuối cùng cho thấy xe cứu thương đang ở trạng thái khẩn cấp, Cloud sẽ gửi lệnh `force_green` để kích hoạt ưu tiên đèn giao thông.

2.3. Các công cụ và thư viện phần mềm hỗ trợ

2.3.1. Python FastAPI cho máy chủ AI

FastAPI là framework web hiện đại của Python, hỗ trợ xây dựng API hiệu năng cao và dễ dàng tích hợp với các mô hình học máy. Trong đề tài, FastAPI được sử dụng để xây dựng máy chủ AI trên AWS EC2, đảm nhận vai trò trung tâm trong việc nhận dữ liệu, xử lý suy luận và trả kết quả cho các thiết bị trong hệ thống.

Các chức năng chính của FastAPI trong hệ thống gồm:

- Tiếp nhận ảnh định kỳ từ ESP32-CAM.
- Tiếp nhận ảnh burst và file âm thanh trong giai đoạn xác minh.
- Gọi các mô hình AI để phân tích dữ liệu.
- Phát lệnh MQTT sau khi có kết quả xác minh.
- Cung cấp API trạng thái và lịch sử sự kiện cho dashboard.

2.3.2. Mô hình YOLOv8 cho nhận diện đối tượng

YOLO (You Only Look Once) là một họ mô hình nhận diện đối tượng thời gian thực có độ chính xác cao và tốc độ xử lý nhanh. Trong đồ án, mô hình YOLOv8 được sử dụng để phát hiện xe cứu thương từ ảnh thu được bởi ESP32-CAM.

YOLOv8 hoạt động bằng cách phân tích trực tiếp toàn bộ ảnh đầu vào và dự đoán đồng thời vị trí cùng nhãn của các đối tượng xuất hiện trong ảnh. Ưu điểm của phương pháp này là tốc độ nhanh, phù hợp với yêu cầu xử lý thời gian thực của hệ thống giao thông thông minh.

2.3.3. Xử lý âm thanh với thư viện Librosa và YAMNet

Librosa là thư viện Python chuyên dùng để xử lý và phân tích tín hiệu âm thanh. Thư viện này hỗ trợ đọc file âm thanh, chuẩn hóa dữ liệu, chuyển đổi tần số mẫu và tiền xử lý tín hiệu trước khi đưa vào mô hình học máy.

YAMNet là mô hình nhận diện âm thanh được Google phát triển, dựa trên dữ liệu AudioSet. Mô hình này có khả năng phân loại nhiều loại âm thanh khác nhau, trong đó có các âm thanh liên quan đến còi xe, còi báo động và các tín hiệu khẩn cấp.

Trong đề tài, Librosa được sử dụng để:

- Đọc file WAV được gửi từ ESP32-CAM.
- Chuẩn hóa tần số mẫu về mức phù hợp với mô hình.
- Tạo dữ liệu đầu vào cho YAMNet.

YAMNet được sử dụng để:

- Phân tích âm thanh môi trường.
- Xác định sự xuất hiện của còi xe cứu thương hoặc âm thanh ưu tiên.
- Kết hợp với kết quả nhận diện hình ảnh để ra quyết định cuối cùng.

2.3.4. MQTT client và thư viện hỗ trợ truyền thông

Để triển khai MQTT trong hệ thống, đề tài sử dụng các thư viện hỗ trợ trên cả máy chủ Python và thiết bị nhúng. Trên máy chủ, thư viện paho-mqtt được dùng để gửi lệnh điều khiển đến ESP32-CAM. Trên thiết bị ESP32-CAM, thư viện tương ứng được sử dụng để đăng ký nhận bản tin từ chủ đề MQTT và thực hiện phản ứng phù hợp.

Bên cạnh đó, các thư viện như json, shutil, boto3, python-multipart và các công cụ hỗ trợ khác cũng được sử dụng để hoàn thiện luồng xử lý dữ liệu, lưu trữ và truyền tải trong hệ thống.

2.3.5. Giao diện giám sát web

Dashboard của hệ thống được xây dựng bằng HTML, CSS và JavaScript nhằm phục vụ mục đích giám sát trạng thái hoạt động, lịch sử nhận diện và kết quả xử lý của hệ thống. Dashboard cho phép người dùng theo dõi:

- Trạng thái server.
- Trạng thái hệ thống.
- Kết quả phát hiện xe cứu thương.
- Kết quả xác minh âm thanh.
- Danh sách sự kiện đã ghi nhận.
- Đường dẫn đến ảnh và âm thanh đã lưu.

Chương II. THIẾT KẾ HỆ THỐNG

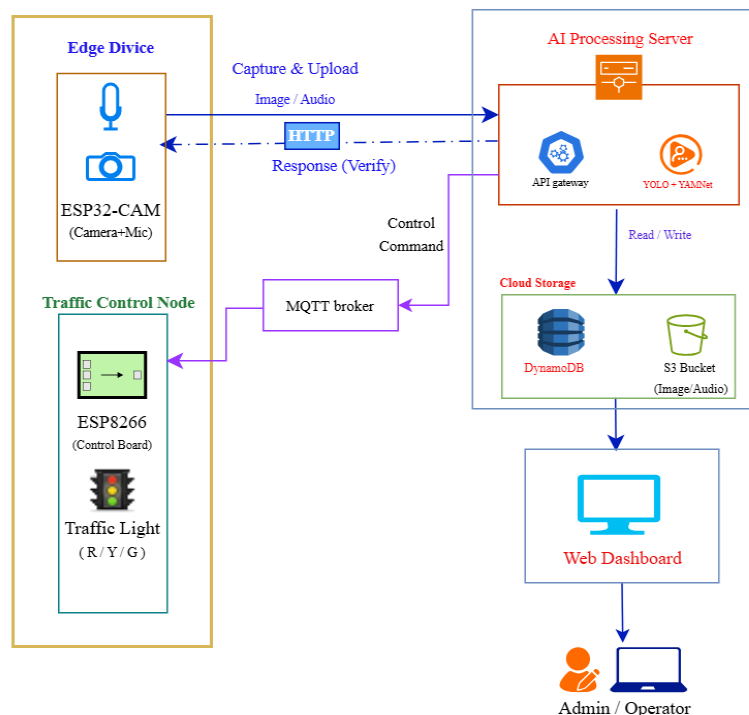
1. Kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình Hybrid Edge-Cloud nhằm kết hợp khả năng xử lý thời gian thực tại thiết bị biên với năng lực xử lý AI trên nền tảng đám mây. ESP32-CAM có nhiệm vụ chụp ảnh giao thông và ghi âm môi trường, sau đó gửi dữ liệu lên Cloud thông qua giao thức HTTP REST API.

Tại Cloud Backend, FastAPI triển khai trên Amazon EC2 thực hiện xử lý dữ liệu bằng mô hình YOLO để nhận diện xe ưu tiên và YAMNet để phát hiện tiếng còi khẩn cấp. Dữ liệu hình ảnh và âm thanh được lưu trữ trên Amazon S3, trong khi DynamoDB lưu metadata và lịch sử hoạt động của hệ thống.

Sau khi xác minh thành công, máy chủ sẽ gửi lệnh MQTT đến ESP8266/ESP32 để điều khiển đèn giao thông ưu tiên cho xe khẩn cấp. Ngoài ra, hệ thống còn tích hợp Dashboard giúp giám sát trạng thái hoạt động và quản lý hệ thống theo thời gian thực.

1.1. Sơ đồ kiến trúc hệ thống



Hình 5. Kiến trúc hệ thống

1.2. Luồng hoạt động hệ thống

Trong trạng thái bình thường, ESP32-CAM thực hiện chụp ảnh giao thông định kỳ và gửi dữ liệu lên AI Server. Máy chủ tiến hành phân tích dữ liệu bằng mô hình YOLO nhằm phát hiện phương tiện ưu tiên.

Khi phát hiện đối tượng nghi ngờ là xe cứu thương hoặc xe ưu tiên, hệ thống chuyển sang giai đoạn xác minh. ESP32-CAM tiếp tục thực hiện burst capture và ghi âm môi trường để thu thập thêm dữ liệu xác thực.

Dữ liệu hình ảnh và âm thanh sau đó được xử lý bởi YOLO và YAMNet. Hệ thống sử dụng cơ chế xác minh đa phương thức để kết hợp kết quả nhận diện hình ảnh và âm thanh trước khi đưa ra quyết định cuối cùng.

Nếu kết quả xác minh hợp lệ, AI Server sẽ publish lệnh `force_green` thông qua MQTT Broker. Bộ điều khiển giao thông (ESP8266) sẽ chuyển hướng ưu tiên sang đèn xanh nhằm tạo điều kiện cho xe khẩn cấp di chuyển.

Sau khi hết thời gian ưu tiên, hệ thống tự động quay về chu kỳ điều khiển đèn giao thông bình thường.

2. Thiết kế tầng Thiết bị biên và Cảm biến

Tầng thiết bị biên là nơi thực hiện các tác vụ thu thập dữ liệu từ môi trường thực tế và phản ứng theo lệnh từ Cloud. Trong hệ thống này, ESP32-CAM đóng vai trò là thiết bị biên trung tâm, kết hợp camera và microphone để thu nhận hai loại dữ liệu đầu vào gồm hình ảnh và âm thanh. Đây là thành phần quyết định khả năng phản hồi thời gian thực của toàn hệ thống.

ESP32-CAM được lập trình hoạt động theo hai chế độ chính: Chế độ bình thường và chế độ xác minh. Ở chế độ bình thường, thiết bị chụp một ảnh định kỳ mỗi giây và gửi ảnh lên máy chủ để phục vụ nhận diện ban đầu. Khi Cloud phát hiện có khả năng xuất hiện xe cứu thương, ESP32-CAM chuyển sang chế độ xác minh và ưu tiên ghi âm môi trường nhằm phục vụ xác minh đa phương thức.

2.1. Cơ chế kích hoạt xác minh dữ liệu hình ảnh (Edge Triggering)

Hệ thống trong đề tài được kích hoạt chủ yếu dựa trên dữ liệu thị giác từ camera. Cơ chế kích hoạt này bắt đầu từ việc ESP32-CAM chụp ảnh định kỳ và gửi ảnh lên Cloud để mô hình YOLOv8 nhận diện đối tượng trong ảnh. Nếu mô hình phát hiện có xe cứu thương với độ tin cậy đạt ngưỡng cấu hình, máy chủ sẽ đánh dấu đây là một candidate thay vì chuyển đèn ngay lập tức.

Sau khi phát hiện candidate, máy chủ phản hồi trạng thái xác minh như `audio_capture` thông qua HTTP response gửi về ESP32-CAM. Khi nhận được phản hồi này, thiết bị tạm dừng chế độ chụp định kỳ thông thường và chuyển sang quy trình xác minh. Cách thiết kế này giúp hệ thống giảm thiểu việc phản ứng sai khi chỉ dựa trên một khung hình đơn lẻ.

Trong firmware của ESP32-CAM, quy trình xác minh được điều khiển trực tiếp dựa trên giá trị action trả về từ Cloud. Khi nhận được action xác minh, thiết bị ưu tiên thực hiện ghi âm môi trường và gửi dữ liệu âm thanh lên máy chủ thay vì tiếp tục chu kỳ chụp ảnh định kỳ.

2.2. Quy trình xác minh âm thanh tại thiết bị biên

Khi chuyển sang trạng thái xác minh, ESP32-CAM kích hoạt microphone INMP441 để ghi lại âm thanh môi trường trong khoảng thời gian ngắn nhằm xác minh sự xuất hiện của còi xe ưu tiên. Quá trình ghi âm được thực hiện thông qua giao tiếp I2S với tần số lấy mẫu 16 kHz và dữ liệu được thu dưới dạng PCM 16-bit mono.

Sau khi hoàn tất quá trình thu âm, firmware tạo phần header WAV trực tiếp trong bộ nhớ RAM và đóng gói toàn bộ dữ liệu âm thanh thành tệp WAV hoàn chỉnh. Tệp này sau đó được gửi lên Cloud bằng giao thức HTTP multipart/form-data để phục vụ phân tích âm thanh bằng mô hình YAMNet.

Thiết kế xử lý trực tiếp trên RAM giúp giảm thao tác đọc ghi bộ nhớ flash và phù hợp với giới hạn tài nguyên của ESP32-CAM. Sau khi upload hoàn tất, firmware giải phóng toàn bộ vùng nhớ tạm để duy trì khả năng hoạt động liên tục của thiết bị.

2.3. Cơ chế điều khiển đèn giao thông

Hệ thống sử dụng ESP8266 làm bộ điều khiển đèn giao thông và nhận lệnh từ Cloud thông qua giao thức MQTT. Firmware được thiết kế theo mô hình finite-state machine (FSM) nhằm bảo đảm quá trình chuyển đổi tín hiệu đèn diễn ra tuần tự và an toàn.

Hệ thống gồm bốn trạng thái chính: `NORMAL_STATE`, `B_YELLOW_STATE`, `EMERGENCY_STATE` và `A_YELLOW_STATE`. Khi nhận lệnh MQTT `force_green`, ESP8266 chuyển từ trạng thái bình thường sang trạng thái vàng chuyển tiếp trước khi kích hoạt đèn xanh ưu tiên cho hướng xe cứu thương. Sau khi hết thời gian ưu tiên, hệ thống tự động quay trở lại trạng thái hoạt động bình thường.

3. Thiết kế tầng Xử lý Đám mây - Cloud Backend

Tầng Cloud là trung tâm xử lý thông minh của hệ thống. Đây là nơi tiếp nhận dữ liệu từ ESP32-CAM, chạy các mô hình AI, lưu trữ kết quả, phát lệnh điều khiển và cung cấp dữ liệu cho dashboard. Trong đề tài hiện tại, Cloud Backend được triển khai trên máy chủ AWS EC2 với FastAPI làm framework chính.

3.1. Luồng xử lý hướng sự kiện của FastAPI Server

FastAPI Server hoạt động theo mô hình hướng sự kiện (Event-Driven), trong đó mỗi yêu cầu gửi từ thiết bị biên sẽ kích hoạt một quy trình xử lý tương ứng. Khi ESP32-CAM gửi ảnh lên endpoint `/upload`, server thực hiện nhận diện đối tượng bằng mô hình YOLOv8 để phát hiện xe cứu thương trong khung hình.

Nếu kết quả nhận diện vượt qua ngưỡng cấu hình, hệ thống không kích hoạt đèn giao thông ngay lập tức mà phản hồi trạng thái xác minh như `audio_capture` hoặc `burst_capture` để yêu cầu thiết bị tiếp tục ghi âm môi trường. Sau khi ESP32-CAM gửi file âm thanh WAV lên endpoint `/audio`, server sử dụng mô hình YAMNet để phân tích tín hiệu âm thanh và xác định sự xuất hiện của còi ưu tiên.

Kết quả từ mô hình hình ảnh và âm thanh sau đó được tổng hợp thông qua cơ chế fusion đa phương thức nhằm đưa ra quyết định cuối cùng. Nếu hệ thống xác nhận

tình huống khẩn cấp, FastAPI Server sẽ publish lệnh MQTT `force_green` đến bộ điều khiển đèn giao thông để kích hoạt chế độ ưu tiên cho xe cứu thương.

3.2. Thiết kế thuật toán phân tích AI kép tại FastAPI Server

Máy chủ AI trong hệ thống sử dụng hai mô hình chính là YOLOv8 và YAMNet. Hai mô hình này đảm nhiệm hai nhiệm vụ khác nhau nhưng bổ trợ cho nhau trong quá trình xác minh phương tiện khẩn cấp.

Ở giai đoạn đầu, YOLOv8 được dùng để nhận diện xe cứu thương từ ảnh định kỳ. Nếu ảnh cho thấy khả năng có xe cứu thương, hệ thống sẽ coi đó là tín hiệu candidate và chuyển sang giai đoạn xác minh sâu hơn. Ở giai đoạn xác minh, hệ thống tiếp tục thu thập thêm dữ liệu để kiểm tra tính ổn định của kết quả nhận diện. Đồng thời, YAMNet được sử dụng để phân tích file âm thanh WAV nhằm xác định sự xuất hiện của còi xe ưu tiên.

Thuật toán phân tích AI kép của hệ thống hoạt động theo logic:

- Nếu YOLO phát hiện xe cứu thương trên ảnh định kỳ nhưng chưa đủ chắc chắn, hệ thống chỉ chuyển sang bước xác minh.
- Nếu dữ liệu xác minh tiếp tục cho thấy khả năng xuất hiện xe cứu thương và YAMNet phát hiện âm thanh còi ưu tiên, hệ thống xác nhận đây là xe cứu thương trong tình huống khẩn cấp.
- Nếu chỉ có hình ảnh xe cứu thương nhưng không có âm thanh ưu tiên, hệ thống không kích hoạt đèn ưu tiên.

3.3. Xây dựng logic xác thực đa yếu tố (Multi-modal Validation)

Thay vì chỉ dựa trên một nguồn dữ liệu, hệ thống kết hợp đồng thời hai kênh thông tin gồm hình ảnh và âm thanh. Logic xác thực đa yếu tố được xây dựng dựa trên ba điều kiện chính:

- Có kết quả nhận diện xe cứu thương đạt ngưỡng xác minh của hệ thống.
- Có tín hiệu âm thanh ưu tiên được phát hiện bởi YAMNet.
- Kết quả tổng hợp vượt qua ngưỡng xác minh của hệ thống.

Khi cả hai điều kiện hình ảnh và âm thanh cùng được thỏa mãn, hệ thống xác nhận `final_detect = true`. Lúc này, máy chủ FastAPI gửi bản tin MQTT `force_green` đến ESP8266 để thay đổi trạng thái đèn giao thông sang chế độ ưu tiên.

Nếu chỉ có hình ảnh xe cứu thương nhưng không phát hiện còi ưu tiên, hệ thống vẫn giữ trạng thái giao thông bình thường nhằm giảm thiểu nguy cơ kích hoạt nhầm.

4. Thiết kế giao diện web giám sát

Dashboard là giao diện web được xây dựng nhằm phục vụ việc giám sát hoạt động của toàn bộ hệ thống theo thời gian thực. Giao diện được triển khai bằng HTML, CSS và JavaScript, trong đó dữ liệu được lấy thông qua các API của FastAPI Server triển khai trên AWS EC2.

Dashboard thực hiện cơ chế cập nhật định kỳ bằng JavaScript để đồng bộ trạng thái mới nhất từ hệ thống Cloud. Các dữ liệu trạng thái và lịch sử sự kiện được FastAPI Server tổng hợp trước khi trả về cho giao diện hiển thị.

Các chức năng chính của Dashboard gồm:

- Hiển thị trạng thái hoạt động của FastAPI Server.
- Hiển thị trạng thái hệ thống như IDLE, VERIFYING, CONFIRMED hoặc COOLDOWN.
- Hiển thị trạng thái điều khiển đèn giao thông và làn ưu tiên hiện tại.
- Hiển thị lịch sử các sự kiện nhận diện xe cứu thương và kết quả xác minh.
- Cho phép xem lại ảnh nhận diện đã lưu trữ trên AWS S3.
- Cho phép phát lại file âm thanh WAV đã lưu trữ trên AWS S3.
- Hỗ trợ cơ chế override thông qua API để phục vụ kiểm thử và điều khiển thủ công hệ thống.

Chương III. TRIỂN KHAI HỆ THỐNG

1. Lắp đặt phần cứng

Hệ thống phần cứng của đề tài được xây dựng theo mô hình mô phỏng một giao lộ giao thông gồm hai làn đường, trong đó đèn giao thông được điều khiển tự động khi phát hiện phương tiện khẩn cấp. Thành phần chính của hệ thống bao gồm ESP32-CAM, microphone INMP441, ESP8266 và cụm đèn LED mô phỏng tín hiệu giao thông.

ESP32-CAM được sử dụng làm thiết bị biên chính, có nhiệm vụ chụp ảnh giao thông và thu âm môi trường khi cần xác minh. Thiết bị này được kết nối với máy tính trong quá trình nạp chương trình, sau đó hoạt động độc lập qua mạng WiFi. Microphone INMP441 được đấu nối theo chuẩn I2S, cho phép thu âm thanh với chất lượng ổn định phục vụ bước phân tích âm thanh trên máy chủ. ESP8266 được sử dụng làm vi điều khiển điều khiển đèn giao thông, nhận lệnh từ hệ thống qua giao thức. Cụm đèn LED được gắn vào các chân xuất tín hiệu của ESP8266 để mô phỏng hai làn đường với ba trạng thái đỏ, vàng và xanh.

Trong quá trình lắp đặt, việc cấp nguồn ổn định cho cả ESP32-CAM và ESP8266 là rất quan trọng, vì thiết bị sẽ đồng thời thực hiện nhiều tác vụ như kết nối WiFi, truyền dữ liệu, xử lý camera và giao tiếp với máy chủ. Nếu nguồn không đủ ổn định, hệ thống có thể xảy ra hiện tượng reset hoặc mất kết nối trong quá trình hoạt động.

2. Lập trình Firmware cho thiết bị biên

Thiết bị biên trong hệ thống bao gồm ESP32-CAM và ESP8266. Mỗi thiết bị đảm nhận một vai trò riêng biệt nhưng phối hợp chặt chẽ với nhau thông qua mạng không dây và giao thức truyền thông đã chọn.

2.1. Mã nguồn ESP32-CAM xử lý Burst Capture và Upload dữ liệu

Khai báo server upload ảnh và audio

```
const char* urlUpload = "http://<IP_SERVER>:8000/upload";  
const char* urlAudio  = "http://<IP_SERVER>:8000/audio";
```

- ESP32-CAM gửi ảnh lên API /upload.
- Khi cần xác minh thêm, thiết bị gửi âm thanh lên API /audio.

Chụp ảnh mỗi 1 giây

```
const unsigned long CAPTURE_INTERVAL_MS = 1000;

if (now - lastCapture >= CAPTURE_INTERVAL_MS)
{
    lastCapture = now;

    String action = uploadFrameTo(urlUpload);

    if (action == "audio_capture" || action == "burst_capture")
    {
        Serial.println("\n[Loop] === BẮT ĐẦU XÁC MINH ÂM THANH ===");

        doRecordAndUploadAudio();

        ensureWiFi();

        reconnectMQTT();

        reinitCamera();

        Serial.println("[Loop] === KẾT THÚC XÁC MINH ===\n");

        lastCapture = millis();
    }
}
```

- ESP32-CAM tự động chụp ảnh định kỳ 1 giây/lần.
- Ảnh được gửi lên server để AI kiểm tra có xe ưu tiên hay không.

Upload ảnh lên server

```
String uploadFrameTo(const char* url);

{
    camera_fb_t* fb = captureWithRetry();

    uint8_t* body = buildMultipart(
```

```

        "image", "frame.jpg", "image/jpeg",
        fb->buf, fb->len,
        totalLen
    );
    int code = http.POST(body, totalLen);
}

```

- Camera chụp ảnh JPEG.
- Ảnh được đóng gói dạng multipart/form-data.
- Sau đó gửi bằng HTTP POST lên server.

Nhận phản hồi từ server

```

if (!deserializeJson(doc, resp) && doc.containsKey("action"))
{
    action = doc["action"].as<String>();
}

```

- ESP32-CAM đọc phản hồi JSON từ server.
- Nếu server yêu cầu "audio_capture" thì thiết bị chuyển sang bước xác minh.

Kích hoạt ghi âm khi nghi ngờ có xe ưu tiên

```

if (action == "audio_capture")
{
    Serial.println("\n[Loop] === BẮT ĐẦU XÁC MINH ÂM THANH ===");
    doRecordAndUploadAudio();
    ensureWiFi();
    reconnectMQTT();
    reinitCamera();
    Serial.println("[Loop] === KẾT THÚC XÁC MINH ===\n");
    lastCapture = millis();
}

```

- Khi AI phát hiện nghi ngờ xe ưu tiên, ESP32-CAM ghi âm để xác minh thêm.
- Sau khi upload audio xong, camera được khởi động lại để tiếp tục chụp ảnh.

Ghi âm 5 giây bằng INMP441

```
#define AUDIO_SAMPLE_RATE 16000
#define AUDIO_DURATION_S 5
while (totalRead < (size_t)AUDIO_TOTAL_BYTES)
{
    esp_err_t err = i2s_read(
        I2S_PORT,
        (uint8_t*)pcmBuf + totalRead,
        chunk,
        &bytesRead,
        pdMS_TO_TICKS(1000)
    );
}
```

- Micro INMP441 ghi âm trong 5 giây.
- Âm thanh được lấy mẫu ở 16kHz, phù hợp để server xử lý âm thanh.

Upload audio lên server

```
uint8_t* body = buildMultipart(
    "audio", "recording.wav", "audio/wav",
    wavBuf, wavTotalLen,
    bodyLen
);

int code = http.POST(body, bodyLen);
```

- Dữ liệu âm thanh được đóng gói thành file WAV.
- Sau đó upload lên API /audio để server xác minh tiếng còi xe ưu tiên.

Lắng nghe MQTT từ server

```
const char* mqtt_topic = "NT532/traffic/light";
if(strcmp(action, "force_green") == 0)
{
    Serial.println("[MQTT] >>> EMERGENCY CONFIRMED — đèn xanh đang bật");
}
```

ESP32-CAM subscribe topic MQTT. Khi server xác nhận có xe ưu tiên, server gửi lệnh "force_green". ESP32-CAM nhận lệnh và ghi log trạng thái xác nhận.

2.2. Mã nguồn ESP8266 điều khiển đèn giao thông và kết nối MQTT

Khai báo MQTT Broker và topic điều khiển

```
MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
MQTT_TOPIC = "NT532/traffic/light"
```

- Server sử dụng MQTT broker để gửi lệnh điều khiển đèn giao thông.
- Topic NT532/traffic/light dùng để giao tiếp giữa server và ESP8266.
- ESP8266 subscribe topic này để nhận lệnh điều khiển trạng thái đèn giao thông.

Hàm publish dữ liệu MQTT

```
def _publish(payload: dict) -> bool:
    try:
        client = mqtt.Client()
        client.connect(MQTT_BROKER, MQTT_PORT, keepalive=60)

        msg_info = client.publish(
            MQTT_TOPIC,
            json.dumps(payload),
            qos=1,      # ← QoS 1: đảm bảo broker nhận được
        )

        # Chờ tối đa 5 giây để message được gửi thật sự
        msg_info.wait_for_publish(timeout=5.0)
```

```
client.disconnect()
```

- Hàm này tạo kết nối MQTT từ server tới broker.
- Dữ liệu được chuyển sang định dạng JSON trước khi gửi.
- qos=1 giúp đảm bảo broker nhận được message.
- Sau khi publish xong, kết nối MQTT được đóng lại.

Hàm gửi lệnh bật đèn xanh ưu tiên

```
def publish_force_green() -> bool:
    return _publish({"action": "force_green"})
```

- Hàm này gửi lệnh force_green.
- Khi ESP8266 nhận được lệnh, hệ thống sẽ bật đèn xanh cho làn ưu tiên.

3. Cấu hình và triển khai Cloud Backend

Hệ thống sử dụng EC2 để chạy máy chủ FastAPI, S3 để lưu trữ dữ liệu đầu vào và DynamoDB để lưu lịch sử nhận diện. Giao tiếp điều khiển giữa Cloud và thiết bị biên được thực hiện qua MQTT Broker công khai.

3.1. Cấu hình AWS EC2

Nhóm sử dụng dịch vụ Amazon EC2 để triển khai cloud server xử lý dữ liệu và giao tiếp với ESP32-CAM. Máy ảo được tạo với hệ điều hành Ubuntu Server và cấu hình Elastic IP để đảm bảo địa chỉ IP public cố định cho hệ thống.

Thông số cấu hình chính:

- Region: us-east-1
- Operating System: Ubuntu Server
- Public Elastic IP: 32.194.73.150
- Security Group mở các cổng:

22/TCP cho SSH

1883/TCP cho MQTT

8000/TCP cho FastAPI API Server

Trên EC2, nhóm tiến hành cài đặt:

```
sudo a apt update  
sudo apt install mosquitto mosquitto-clients -y
```

Cấu hình MQTT Broker:

```
listener 1883  
allow_anonymous true
```

Khởi động MQTT service:

```
sudo systemctl enable mosquitto  
sudo systemctl start mosquitto
```

EC2 được sử dụng để:

- Chạy FastAPI server.
- Chạy MQTT Broker Mosquitto.
- Nhận ảnh từ ESP32-CAM.
- Thực hiện nhận diện bằng YOLO.
- Gửi lệnh điều khiển ngược về ESP8266 thông qua MQTT.

3.2. Cấu hình Amazon S3

Amazon S3 được sử dụng để lưu trữ ảnh từ ESP32-CAM sau khi hệ thống xử lý nhận diện.

Nhóm tạo một S3 Bucket để:

- Lưu ảnh capture từ ESP32-CAM.
- Lưu ảnh sau khi xử lý YOLO.
- Tạo URL phục vụ dashboard giám sát.

EC2 sử dụng thư viện boto3 để upload ảnh lên S3:

```
import boto3
```

Thông tin AWS được cấu hình bằng biến môi trường:

```
AWS_ACCESS_KEY_ID=...  
AWS_SECRET_ACCESS_KEY=...
```

```
AWS_REGION=us-east-1
S3_BUCKET=nt532-nhom08-captures
API_KEY=nhom08
MQTT_BROKER=localhost
MQTT_PORT=1883
```

3.3. Cấu hình Amazon DynamoDB

DynamoDB được sử dụng để lưu metadata và log sự kiện của hệ thống.

Bảng DynamoDB lưu các thông tin:

- Timestamp phát hiện xe ưu tiên.
- Trạng thái YOLO detection.
- URL ảnh trên S3.
- Trạng thái đèn giao thông.
- Log cảnh báo hệ thống.

EC2 kết nối DynamoDB thông qua thư viện:

```
import boto3
```

DynamoDB được lựa chọn do:

- Hỗ trợ dữ liệu thời gian thực.
- Tốc độ đọc/ghi nhanh.
- Khả năng mở rộng tốt cho hệ thống IoT.

4. Máy chủ AI (FastAPI AI Server)

Máy chủ AI là trung tâm xử lý của toàn bộ hệ thống. Đây là nơi tiếp nhận dữ liệu, thực hiện suy luận bằng mô hình AI, ra quyết định xác minh và điều phối các hành động tiếp theo.

4.1. Huấn luyện mô hình AI nhận diện xe cứu thương

4.1.1. Xây dựng tập dữ liệu

Bộ dữ liệu được xây dựng phục vụ bài toán nhận diện xe cứu thương trong môi trường giao thông. Ban đầu, tập dữ liệu gồm 500 ảnh xe cứu thương được thu thập từ nhiều nguồn khác nhau trên Internet. Để tăng tính đa dạng và cải thiện khả năng tổng quát hóa của mô hình, tập dữ liệu này tiếp tục được kết hợp với các bộ dữ liệu giao thông công khai chứa hình ảnh xe cứu thương và các phương tiện giao thông khác.

Sau khi tổng hợp dữ liệu, toàn bộ ảnh được chuẩn hóa và gán nhãn theo định dạng YOLO. Bộ dữ liệu gồm hai lớp đối tượng:

- ambulance: xe cứu thương.
- not_ambulance: các phương tiện giao thông thông thường.

Nhằm tăng số lượng mẫu huấn luyện và mô phỏng các điều kiện quan sát thực tế từ camera giao thông, kỹ thuật tăng cường dữ liệu (Data Augmentation) được áp dụng. Cụ thể, các ảnh được xử lý bằng phương pháp làm mờ (Blur Augmentation) thông qua việc giảm độ phân giải, nội suy lại kích thước ảnh và áp dụng bộ lọc Gaussian Blur. Sau quá trình mở rộng và xử lý dữ liệu, bộ dữ liệu cuối cùng gồm 1.900 ảnh, trong đó:

Loại dữ liệu	Số lượng ảnh
Ambulance	1.107
Not Ambulance	791
Tổng cộng	1.900

Dữ liệu được chia thành hai tập:

Tập dữ liệu	Số lượng ảnh	Tỷ lệ
Training Set	1.688	88,84%
Validation Set	212	11,16%
Tổng cộng	1.900	100%

Việc kết hợp dữ liệu từ nhiều nguồn cùng với kỹ thuật tăng cường dữ liệu giúp mô hình học được nhiều đặc trưng khác nhau của xe cứu thương trong các điều kiện giao thông đa dạng, từ đó nâng cao độ chính xác và khả năng hoạt động trong môi trường thực tế.

Cấu hình dữ liệu được khai báo trong file data.yaml:

```
path: [...]
train: train/images
val: valid/images
nc: 2
names:
  0: ambulance
  1: not_ambulance
```

Dữ liệu được chia thành tập huấn luyện (train) và tập kiểm tra (validation) nhằm đánh giá khả năng tổng quát hóa của mô hình trong quá trình học.

4.1.2. Huấn luyện mô hình YOLOv8

Mô hình YOLOv8 được huấn luyện theo phương pháp Transfer Learning. Sau giai đoạn huấn luyện ban đầu, quá trình fine-tuning được thực hiện bằng cách sử dụng trọng số tốt nhất (best.pt) thu được từ lần huấn luyện trước nhằm cải thiện khả năng nhận diện xe cứu thương trong các điều kiện giao thông khác nhau.

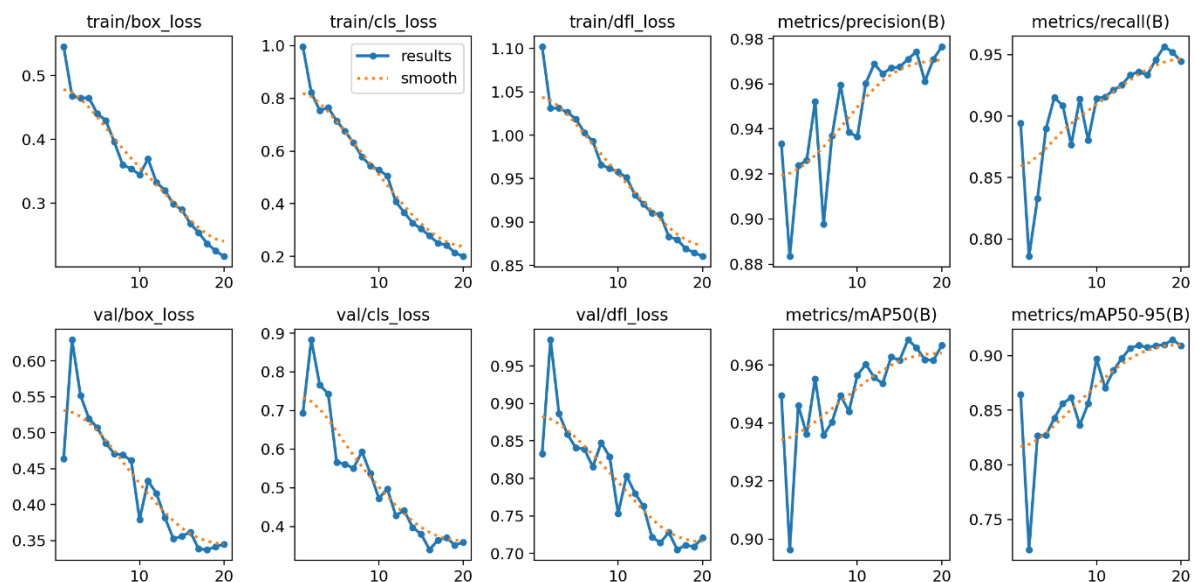
Quá trình huấn luyện được thực hiện với các tham số:

```
from ultralytics import YOLO
model = [...]
model.train(
    data="data.yaml",
    epochs=10,
    imgsz=640,
```

```
batch=8,
lr=0.0005,
augment=True
```

Trong quá trình huấn luyện, cơ chế Data Augmentation được kích hoạt nhằm tạo ra nhiều biến thể dữ liệu khác nhau thông qua các phép biến đổi như xoay ảnh, thay đổi độ sáng, lật ảnh và thay đổi tỷ lệ đối tượng. Điều này giúp cải thiện khả năng tổng quát hóa của mô hình trong các điều kiện giao thông thực tế.

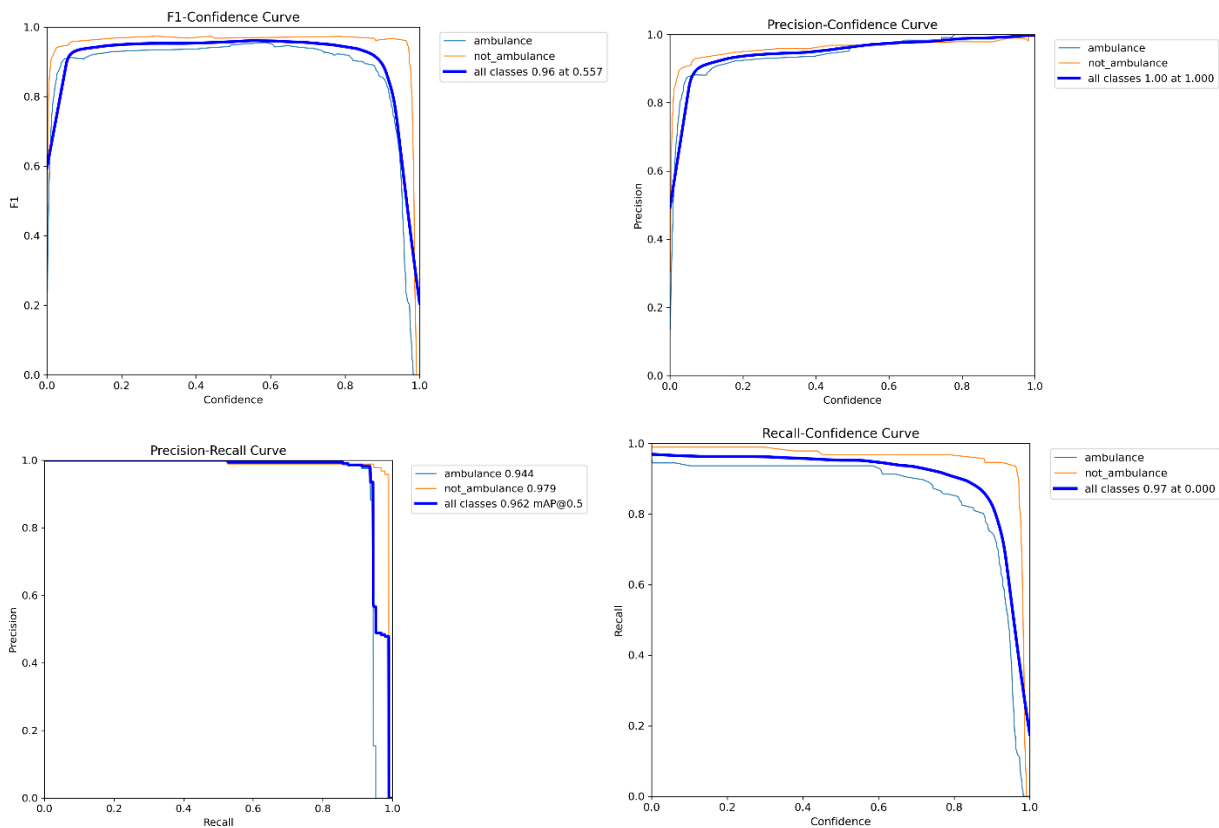
4.1.3. Kết quả huấn luyện



Hình 6. Tiến trình huấn luyện

Các hàm mất mát (Loss): box_loss (lỗi định vị khung bao), cls_loss (lỗi phân loại lớp), và dfl_loss. Cả hai hàng (huấn luyện và kiểm thử) đều giảm dần ổn định, chứng tỏ mô hình đang học rất tốt và không bị hiện tượng quá khớp (overfitting).

Các chỉ số đánh giá (Metrics): precision (độ chính xác), recall (độ bao phủ), mAP50 và mAP50-95 đều tăng dần và đạt đỉnh ở các epoch cuối (đều vượt ngưỡng 0.9 - tức trên 90%). Điều này cho thấy mô hình càng ngày càng thông minh qua các vòng lặp.



Hình 7. Các đường cong hiệu năng (F1, P, R, PR Curves)

Kết quả đánh giá mô hình được thể hiện thông qua các đường cong F1-Score, Precision, Recall, Precision-Recall và chỉ số mAP.

Đường cong F1-Score phản ánh sự cân bằng giữa Precision và Recall thông qua trung bình điều hòa của hai chỉ số này. Kết quả cho thấy mô hình đạt giá trị F1-Score cao nhất là 0.96 tại ngưỡng độ tin cậy (Confidence Threshold) bằng 0.557. Điều này cho thấy đây là ngưỡng phù hợp để đạt được sự cân bằng tối ưu giữa khả năng phát hiện đối tượng và độ chính xác của dự đoán.

Đường cong Precision thể hiện tỷ lệ dự đoán đúng trên tổng số dự đoán dương tính của mô hình. Khi ngưỡng độ tin cậy được tăng lên, Precision cũng tăng tương ứng và đạt giá trị tối đa 1.00 ở các mức độ tin cậy cao. Kết quả này cho thấy các dự đoán được đưa ra ở mức độ tin cậy cao hầu như không xuất hiện trường hợp nhận diện sai.

Đường cong Recall thể hiện khả năng phát hiện đầy đủ các đối tượng thực tế có trong ảnh. Khi ngưỡng độ tin cậy ở mức thấp, Recall đạt khoảng 0.97, cho thấy mô hình có khả năng phát hiện gần như toàn bộ đối tượng cần nhận diện. Tuy nhiên, khi tăng ngưỡng độ tin cậy, Recall giảm dần do mô hình trở nên khắt khe hơn trong việc đưa ra dự đoán, dẫn đến một số đối tượng có thể bị bỏ sót.

Biểu đồ Precision-Recall mô tả mối quan hệ đánh đổi giữa hai chỉ số Precision và Recall. Đường cong có xu hướng áp sát góc trên bên phải của đồ thị, cho thấy mô hình duy trì được độ chính xác cao đồng thời vẫn đảm bảo khả năng phát hiện đối tượng hiệu quả. Điều này phản ánh chất lượng nhận diện tốt của mô hình trên tập dữ liệu kiểm thử.

Chỉ số $mAP@0.5$ đạt 0.962 (96.2%), cho thấy mô hình có độ chính xác cao trong nhiệm vụ nhận diện đối tượng. Trong đó, lớp `not_ambulance` đạt giá trị mAP là 0.979, trong khi lớp `ambulance` đạt 0.944. Các kết quả này chứng minh mô hình có khả năng phân biệt hiệu quả giữa xe cứu thương và các phương tiện giao thông thông thường, đáp ứng yêu cầu triển khai trong hệ thống nhận diện xe ưu tiên thời gian thực.

4.1.4. Tích hợp mô hình nhận diện

Mô hình YOLOv8 sau khi huấn luyện được tích hợp vào FastAPI Server để thực hiện suy luận thời gian thực. Thiết bị ESP32-CAM định kỳ gửi ảnh đến endpoint `/upload`, sau đó máy chủ sử dụng mô hình `best.pt` để phát hiện xe cứu thương trong khung hình.

Nếu kết quả nhận diện có độ tin cậy vượt ngưỡng cấu hình, hệ thống không kích hoạt đèn ưu tiên ngay lập tức mà đánh dấu đây là một trường hợp nghi vấn (candidate). Máy chủ phản hồi lệnh `audio_capture` để yêu cầu ESP32-CAM thu thập thêm dữ liệu âm thanh phục vụ bước xác minh đa phương thức.

4.2. Xây dựng khối xử lý âm thanh còi ưu tiên

Khối xử lý âm thanh được xây dựng dựa trên file WAV do ESP32-CAM gửi lên endpoint `/audio`. Máy chủ sử dụng thư viện Librosa để đọc và chuẩn hóa dữ liệu âm thanh trước khi đưa vào mô hình YAMNet để phân loại âm thanh môi trường.

Mô hình YAMNet được sử dụng để phát hiện các tín hiệu liên quan đến còi xe ưu tiên và âm thanh khẩn cấp. Nếu hệ thống phát hiện âm thanh còi ưu tiên với độ tin cậy đạt ngưỡng, kết quả này sẽ được kết hợp với dữ liệu hình ảnh nhằm phục vụ bước xác minh cuối cùng.

Việc bổ sung xác minh bằng âm thanh giúp hệ thống phân biệt giữa xe cứu thương thông thường và xe cứu thương đang thực hiện nhiệm vụ khẩn cấp.

4.3. Lập trình logic xác thực đa yếu tố

Hệ thống áp dụng cơ chế xác thực đa yếu tố bằng cách kết hợp đồng thời dữ liệu hình ảnh và âm thanh thay vì chỉ sử dụng một nguồn dữ liệu duy nhất.

Luồng xử lý được triển khai như sau:

- ESP32-CAM gửi ảnh định kỳ lên FastAPI Server.
- YOLOv8 thực hiện nhận diện xe cứu thương từ ảnh đầu vào.
- Nếu phát hiện candidate, server phản hồi lệnh `audio_capture`.
- ESP32-CAM ghi âm môi trường và gửi file WAV lên endpoint `/audio`.
- FastAPI Server sử dụng YAMNet để phân tích tín hiệu âm thanh.
- Nếu kết quả xác minh đạt yêu cầu, hệ thống xác nhận `final_detect = true`.
- Server gửi lệnh MQTT `force_green` đến ESP8266 để kích hoạt chế độ đèn ưu tiên.

5. Lập trình giao diện Web Dashboard

Dashboard được xây dựng như một giao diện giám sát trực quan cho toàn bộ hệ thống. Giao diện sử dụng HTML, CSS và JavaScript thuần, kết nối trực tiếp với các API do FastAPI cung cấp. Dashboard có chức năng hiển thị trạng thái server, trạng thái hệ thống, lịch sử sự kiện nhận diện, ảnh đã lưu và âm thanh đã lưu.

Dữ liệu trên dashboard được cập nhật định kỳ từ các endpoint `/status` và `/events`. Các bản ghi lịch sử được đọc từ DynamoDB thông qua API của máy chủ, sau đó hiển thị dưới dạng bảng để người dùng dễ theo dõi. Khi người dùng nhấn vào liên kết “View”, ảnh được mở trực tiếp dưới dạng cửa sổ xem trước, giúp quá trình kiểm tra trở nên trực quan hơn.

Chương IV. MÔ PHỎNG

1. Kịch bản 1: Không có xe cứu thương

Trong kịch bản này, ESP32-CAM chụp ảnh giao thông theo chu kỳ và gửi dữ liệu lên máy chủ AWS EC2 thông qua FastAPI Server. Mô hình YOLOv8 không phát hiện xe cứu thương trong ảnh nên hệ thống không kích hoạt bước xác minh âm thanh và cũng không gửi lệnh ưu tiên đèn giao thông.

Đèn giao thông tiếp tục hoạt động theo chu kỳ bình thường. Kịch bản này được sử dụng để kiểm tra khả năng duy trì trạng thái ổn định của hệ thống khi không có phương tiện ưu tiên xuất hiện.

2. Kịch bản 2: Phát hiện nghi ngờ xe cứu thương nhưng xác minh âm thanh không đạt

Ở kịch bản này, ESP32-CAM gửi ảnh lên FastAPI Server và mô hình YOLOv8 phát hiện đối tượng có khả năng là xe cứu thương. Sau đó hệ thống chuyển sang trạng thái xác minh và phản hồi lệnh `audio_capture` để yêu cầu ESP32-CAM ghi âm môi trường bằng microphone INMP441.

Cloud tiếp tục phân tích dữ liệu âm thanh bằng mô hình YAMNet. Nếu kết quả không phát hiện còi ưu tiên hoặc độ tin cậy không đạt ngưỡng xác minh, hệ thống sẽ hủy trạng thái ưu tiên và không kích hoạt điều khiển đèn giao thông.

Hệ thống chỉ ghi nhận sự kiện nghi ngờ, lưu log và dữ liệu lên S3/DynamoDB nhưng không chuyển sang chế độ đèn ưu tiên. Đèn giao thông vẫn duy trì chu kỳ hoạt động bình thường. Kịch bản này giúp đánh giá khả năng giảm nhận diện sai của hệ thống.

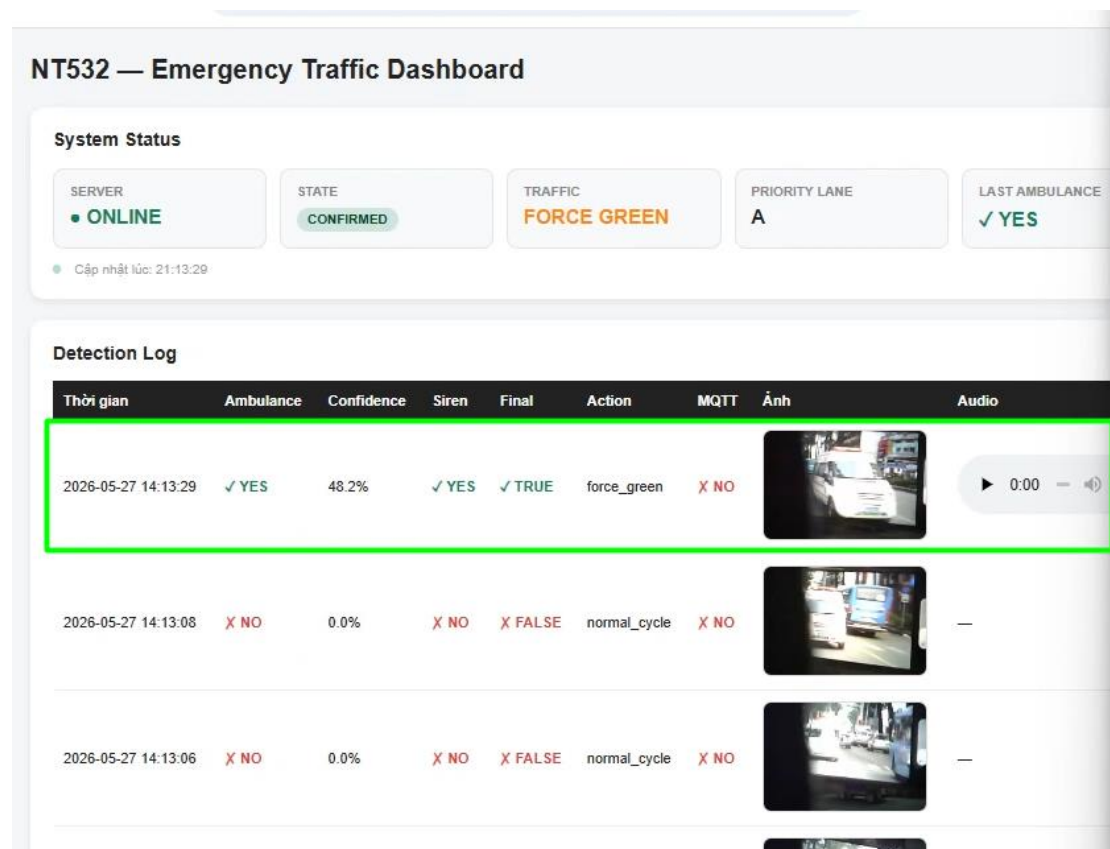
3. Kịch bản 3: Phát hiện và xác minh thành công xe cứu thương

Trong kịch bản này, ESP32-CAM gửi ảnh giao thông lên FastAPI Server và mô hình YOLOv8 phát hiện xe cứu thương với độ tin cậy đạt ngưỡng cấu hình. Hệ thống tiếp tục yêu cầu ESP32-CAM ghi âm môi trường để thực hiện bước xác minh bổ sung.

Cloud tiến hành phân tích âm thanh bằng mô hình YAMNet. Khi cả dữ liệu hình ảnh và âm thanh đều xác nhận có xe cứu thương trong trạng thái khẩn cấp, hệ thống xác

nhận `final_detect = true` và gửi lệnh MQTT `force_green` đến ESP8266 điều khiển đèn giao thông.

Đèn giao thông chuyển sang trạng thái ưu tiên trong khoảng thời gian cấu hình, sự kiện được lưu vào DynamoDB, dữ liệu ảnh và âm thanh được lưu trên Amazon S3, đồng thời dashboard hiển thị trạng thái xác minh và ưu tiên thành công.



Hình 8. Hệ thống nhận diện đúng xe cứu thương và dashboard hiển thị trạng thái CONFIRMED

Chương V. TỔNG KẾT

1. Kết quả đạt được

Đề tài đã xây dựng thành công một hệ thống mô phỏng có khả năng phát hiện và xác minh xe cứu thương theo thời gian thực thông qua dữ liệu hình ảnh và âm thanh. Hệ thống được triển khai theo mô hình phân tầng gồm thiết bị biên, máy chủ xử lý AI và bộ điều khiển đèn giao thông, bảo đảm đúng với mục tiêu thiết kế ban đầu.

Về phần cứng, nhóm đã triển khai thành công ESP32-CAM để thu nhận hình ảnh giao thông và microphone INMP441 để thu âm môi trường. ESP8266 được sử dụng để điều khiển cụm đèn LED mô phỏng giao lộ thông minh. Các thiết bị có khả năng kết nối WiFi ổn định và giao tiếp với máy chủ thông qua giao thức mạng.

Về phần mềm và xử lý AI, hệ thống đã:

- Xây dựng máy chủ FastAPI chạy trên AWS EC2.
- Triển khai thành công mô hình YOLOv8 để nhận diện xe cứu thương từ ảnh camera.
- Triển khai mô hình YAMNet để phân tích âm thanh còi ưu tiên từ file WAV.
- Xây dựng cơ chế xác minh đa phương thức kết hợp hình ảnh và âm thanh.
- Thiết kế cơ chế xác minh hai bước giúp giảm thiểu sai sót nhận diện.
- Xây dựng luồng xử lý thời gian thực thông qua MQTT.

Hệ thống hiện tại đã có khả năng hoạt động theo đúng luồng xử lý thiết kế:

- ESP32-CAM chụp ảnh định kỳ và gửi lên Cloud.
- Cloud thực hiện nhận diện nhanh bằng YOLOv8.
- Khi phát hiện candidate, Cloud gửi lệnh xác minh.
- ESP32-CAM thực hiện burst capture và ghi âm.
- Máy chủ xác minh lại bằng YOLOv8 và YAMNet.
- Nếu kết quả xác nhận xe cứu thương đang hoạt động khẩn cấp, hệ thống sẽ kích hoạt đèn ưu tiên.

Ngoài ra, nhóm cũng đã xây dựng dashboard giám sát cho phép theo dõi trạng thái hoạt động của hệ thống, lịch sử nhận diện, ảnh và âm thanh đã lưu, đồng thời hỗ trợ điều khiển thủ công phục vụ kiểm thử.

Nhìn chung, đề tài đã chứng minh được tính khả thi của việc kết hợp AI, IoT và điện toán đám mây trong bài toán giao thông thông minh.

2. Hạn chế và khó khăn

Mặc dù đã đạt được các mục tiêu chính đề ra, hệ thống vẫn còn tồn tại một số hạn chế trong quá trình triển khai và thử nghiệm.

Trước hết, ESP32-CAM là thiết bị có tài nguyên phần cứng hạn chế, đặc biệt về bộ nhớ RAM và khả năng xử lý đa nhiệm. Trong quá trình phát triển, việc kết hợp camera, WiFi, MQTT, SPIFFS và microphone I2S đồng thời trên cùng một thiết bị gây ra nhiều vấn đề như reset ngẫu nhiên, thiếu bộ nhớ hoặc mất ổn định khi xử lý dữ liệu lớn. Vì vậy, nhóm phải tối ưu lại thời lượng ghi âm, số lượng burst ảnh và cách quản lý bộ nhớ để hệ thống có thể hoạt động ổn định.

Bên cạnh đó, chất lượng nhận diện âm thanh còn phụ thuộc nhiều vào môi trường thực tế. Trong điều kiện có nhiều tiếng ồn hoặc âm thanh giao thông phức tạp, mô hình YAMNet có thể gặp khó khăn trong việc phân biệt chính xác còi ưu tiên. Khoảng cách giữa xe cứu thương và microphone cũng ảnh hưởng đáng kể đến chất lượng thu âm.

Về xử lý hình ảnh, mô hình YOLOv8 hiện tại hoạt động tốt trong điều kiện ánh sáng tương đối ổn định, tuy nhiên độ chính xác có thể giảm trong môi trường thiếu sáng, thời tiết xấu hoặc góc camera không thuận lợi. Ngoài ra, tập dữ liệu huấn luyện hiện tại vẫn còn giới hạn nên hệ thống chưa được đánh giá đầy đủ trên nhiều tình huống giao thông khác nhau.

Một hạn chế khác là hệ thống hiện vẫn phụ thuộc vào kết nối Internet để gửi dữ liệu lên Cloud và nhận lệnh điều khiển. Nếu mạng không ổn định hoặc độ trễ tăng cao, thời gian phản hồi của hệ thống có thể bị ảnh hưởng.

Ngoài ra, hệ thống mới dừng lại ở mức mô hình thử nghiệm, chưa được triển khai trên giao lộ thực tế với nhiều làn xe, nhiều camera và lưu lượng phương tiện lớn. Một số chức năng bảo mật nâng cao như xác thực MQTT bằng chứng chỉ số hoặc mã hóa toàn bộ dữ liệu truyền tải cũng chưa được triển khai đầy đủ.

3. Hướng cải tiến và phát triển

Hệ thống có thể được nâng cấp bằng cách sử dụng các thiết bị Edge AI mạnh hơn như Jetson Nano hoặc Raspberry Pi kết hợp TPU để thực hiện một phần xử lý AI

trực tiếp tại thiết bị biên, giúp giảm độ trễ truyền dữ liệu lên Cloud và tăng khả năng phản hồi theo thời gian thực.

Về nhận diện hình ảnh, có thể mở rộng tập dữ liệu huấn luyện và fine-tune lại mô hình YOLOv8 để cải thiện độ chính xác trong các điều kiện môi trường khác nhau như ban đêm, trời mưa hoặc giao thông đông đúc. Đồng thời, hệ thống cũng có thể được mở rộng để nhận diện thêm các loại phương tiện ưu tiên khác như xe cứu hỏa hoặc xe cảnh sát.

Đối với xử lý âm thanh, có thể nghiên cứu áp dụng các kỹ thuật lọc nhiễu hoặc các mô hình âm thanh chuyên biệt hơn để nâng cao khả năng nhận diện còi ưu tiên trong môi trường thực tế phức tạp. Ngoài ra, việc tăng chất lượng microphone hoặc sử dụng nhiều microphone định hướng cũng là một hướng cải tiến phù hợp.

Về hạ tầng Cloud, hệ thống có thể được mở rộng bằng cách sử dụng các dịch vụ MQTT bảo mật chuyên dụng như AWS IoT Core thay cho broker công khai nhằm tăng tính ổn định và bảo mật. Bên cạnh đó, có thể bổ sung cơ chế phân quyền người dùng, mã hóa dữ liệu và giám sát hệ thống theo thời gian thực.

Triển khai thử nghiệm trên môi trường giao thông thực tế nhằm đánh giá toàn diện hơn về độ chính xác, độ ổn định và khả năng vận hành trong các tình huống thực tế.

PHỤ LỤC

1. Bảng phân công công việc

STT	Thành viên	Nội dung công việc thực hiện
23521837	Trần Ngọc Thuý Vy	– Thiết kế và lập trình firmware cho ESP32-CAM và ESP8266. – Tích hợp camera OV2640 và microphone INMP441. – Xây dựng cơ chế chụp ảnh định kỳ, ghi âm và gửi dữ liệu lên Cloud. – Lập trình điều khiển đèn giao thông theo mô hình. – Tích hợp MQTT client/subscriber cho ESP32-CAM và ESP8266.
23521768	Bùi Nguyễn Tường Vân	– Triển khai hệ thống trên AWS EC2. – Cấu hình FastAPI Server và MQTT Broker Mosquitto. – Xây dựng API xử lý dữ liệu từ ESP32-CAM. – Tích hợp Amazon S3 và DynamoDB để lưu trữ dữ liệu hệ thống. – Thiết kế dashboard giám sát.
23520425	Đỗ Gia Hân	– Huấn luyện và tích hợp mô hình YOLOv8 nhận diện xe cứu thương. – Tích hợp mô hình YAMNet xử lý âm thanh còi. – Xây dựng thuật toán xác thực đa phương thức. – Tối ưu ngưỡng nhận diện và logic xác minh khẩn cấp. – Đánh giá kết quả thực nghiệm và độ chính xác của hệ thống AI.

2. Mã nguồn hệ thống

Mã nguồn của đề tài được quản lý và lưu trữ tại: [Nhom08-NT532.Q21](#)

NGUỒN THAM KHẢO

- [1] Espressif Systems, ESP32 Series Datasheet.
- [2] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” CVPR 2016.
- [3] TensorFlow Team, “Sound classification with YAMNet.”
- [4] Sebastián Ramírez, FastAPI Documentation.
- [5] OASIS, MQTT Version 3.1.1 Standard.
- [6] Amazon Web Services, Amazon EC2 Documentation.
- [7] B. McFee et al., “librosa: Audio and Music Signal Analysis in Python,” SciPy 2015.